We'll come back to these concepts again from different angles throughout the book, and our goal here is just to give you some initial exposure.

2.1 Introducing Scala the language: an example

The following is a complete program listing in Scala, which we'll talk through. We aren't introducing any new concepts of functional programming here. Our goal is just to introduce the Scala language and its syntax.

Listing 2.1 A simple Scala program

```
// A comment!
/* Another comment */
/** A documentation comment */
object MyModule {
  def abs(n: Int): Int =
    if (n < 0) -n
    else n

  private def formatAbs(x: Int) = {
    val msg = "The absolute value of %d is %d"
    msg.format(x, abs(x))
  }

  def main(args: Array[String]): Unit =
    println(formatAbs(-42))
}
```

Declares a singleton object, which simultaneously declares a class and its only instance.

`abs` takes an integer and returns an integer.

Returns the negation of `n` if it's less than zero.

A private method can only be called by other members of `MyModule`.

A string with two placeholders for numbers marked as `%d`.

Replaces the two `%d` placeholders in the string with `x` and `abs(x)` respectively.

`Unit` serves the same purpose as `void` in languages like Java or C.

We declare an object (also known as a *module*) named `MyModule`. This is simply to give our code a place to live and a name so we can refer to it later. Scala code has to be in an `object` or a `class`, and we use an `object` here because it's simpler. We put our code inside the object, between curly braces. We'll discuss objects and classes in more detail shortly. For now, we'll just look at this particular object.

The `MyModule` object has three *methods*, introduced with the `def` keyword: `abs`, `formatAbs`, and `main`. We'll use the term *method* to refer to some function or field defined within an object or class using the `def` keyword. Let's now go through the methods of `MyModule` one by one.

The object keyword

The `object` keyword creates a new *singleton type*, which is like a `class` that only has a single named instance. If you're familiar with Java, declaring an `object` in Scala is a lot like creating a new instance of an anonymous class.

Scala has no equivalent to Java's `static` keyword, and an `object` is often used in Scala where you might use a class with static members in Java.

The `abs` method is a pure function that takes an integer and returns its absolute value:

```
def abs(n: Int): Int =
  if (n < 0) -n
  else n
```

The `def` keyword is followed by the name of the method, which is followed by the parameter list in parentheses. In this case, `abs` takes only one argument, `n` of type `Int`. Following the closing parenthesis of the argument list, an optional type annotation (the `: Int`) indicates that the type of the result is `Int` (the colon is pronounced "has type").

The body of the method itself comes after a single equals sign (`=`). We'll sometimes refer to the part of a declaration that goes before the equals sign as the *left-hand side* or *signature*, and the code that comes after the equals sign as the *right-hand side* or *definition*. Note the absence of an explicit `return` keyword. The value returned from a method is simply whatever value results from evaluating the right-hand side. All expressions, including `if` expressions, produce a result. Here, the right-hand side is a single expression whose value is either `-n` or `n`, depending on whether `n < 0`.

The `formatAbs` method is another pure function:

```
private def formatAbs(x: Int) = {
  val msg = "The absolute value of %d is %d."
  msg.format(x, abs(x))
}
```

`format` is a standard library method defined on `String`

Here we're calling the `format` method on the `msg` object, passing in the value of `x` along with the value of `abs` applied to `x`. This results in a new string with the occurrences of `%d` in `msg` replaced with the evaluated results of `x` and `abs(x)`, respectively.

This method is declared `private`, which means that it can't be called from any code outside of the `MyModule` object. This function takes an `Int` and returns a `String`, but note that the return type is not declared. Scala is usually able to infer the return types of methods, so they can be omitted, but it's generally considered good style to explicitly declare the return types of methods that you expect others to use. This method is private to our module, so we can omit the type annotation.

The body of the method contains more than one statement, so we put them inside curly braces. A pair of braces containing statements is called a *block*. Statements are separated by newlines or by semicolons. In this case, we're using a newline to separate our statements, so a semicolon isn't necessary.

The first statement in the block declares a `String` named `msg` using the `val` keyword. It's simply there to give a name to the string value so we can refer to it again. A `val` is an immutable variable, so inside the body of the `formatAbs` method the name `msg` will always refer to the same `String` value. The Scala compiler will complain if you try to reassign `msg` to a different value in the same block.

Remember, a method simply returns the value of its right-hand side, so we don't need a `return` keyword. In this case, the right-hand side is a block. In Scala, the value